

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Applied Business Analytics Practical

Practical – XI

Roll No.: T005	Name: Ritesh Chaubey
Class: TYIT	Batch: 1
Date of Assignment: 03-10-2026	Date/Time of Submission: 03-10-2026

AIM : Optimization Modeling and Simulation Modeling:

A) Formulate an optimization model (e.g., linear programming, integer programming) to solve a real-world business problem and analyze the optimal solution.

Part A: Optimization Modeling

Business Problem: A company produces two products, Product A and Product B. The company wants to maximize its profit using the available machine and labour hours.

Given data:

Product	Profit Per Unit	Machine Hours	Labour Hours
A	40	2	1
B	30	1	2

Available Machine Hours = 100

Available Labour Hours = 80

Practical Questions:

1. Define the decision variables for Product A and Product B.

Let:

- A = number of units of Product A
- B = number of units of Product B

2. Define the objective of the problem.

The objective is to maximize the total profit from producing Product A and Product B.

3. Formulate the objective function to maximize the total profit.

Profit from A = ₹40 per unit

Profit from B = ₹30 per unit

Therefore:

Maximize Profit = $40A + 30B$

4. Formulate the machine-hour constraint.

Product A requires 2 machine hours.

Product B requires 1 machine hour.

Available machine hours = 100.

Therefore:

$2A + B \leq 100$

5. Formulate the labour-hour constraint.

Product A requires 1 labour hour.

Product B requires 2 labour hours.

Available labour hours = 80.

Therefore:

$$A + 2B \leq 80$$

6. Add non-negative constraints for the decision variables.

The number of products cannot be negative.

Therefore:

$$A \geq 0$$

$$B \geq 0$$

7. Solve the linear programming problem using Python.

CODE :-

```
from scipy.optimize import linprog
# Objective coefficients
# linprog performs minimization, so use negative values
profit = [-40, -30]
# Constraints
A = [
    [2, 1], # Machine hours
    [1, 2]  # Labour hours
]
# Available resources
b = [100, 80]
# Non-negative constraints
bounds = [
    (0, None), # Product A
    (0, None)  # Product B
]
result = linprog(
    profit,
    A_ub=A,
    b_ub=b,
    bounds=bounds,
    method="highs"
)
print(result)
```

OUTPUT :-

```

message: Optimization terminated successfully. (HiGS Status 7: Optimal)
success: True
status: 0
  fun: -2200.0
    x: [ 4.000e+01  2.000e+01]
  nit: 2
 lower: residual: [ 4.000e+01  2.000e+01]
      marginals: [ 0.000e+00  0.000e+00]
 upper: residual: [          inf          inf]
      marginals: [ 0.000e+00  0.000e+00]
 eqlin: residual: []
      marginals: []
 ineqlin: residual: [ 0.000e+00  0.000e+00]
       marginals: [-1.667e+01 -6.667e+00]
 mip_node_count: 0
 mip_dual_bound: 0.0
  mip_gap: 0.0

```

8. Find the optimal number of units of Product A and Product B.

CODE :-

```

product_A = result.x[0]
product_B = result.x[1]
print("Optimal units of Product A:", product_A)
print("Optimal units of Product B:", product_B)

```

OUTPUT :-

```

Optimal units of Product A: 40.0
Optimal units of Product B: 20.0

```

9. Calculate the maximum profit.

CODE :-

```

maximum_profit = (
    40 * product_A +
    30 * product_B
)
print("Maximum Profit = ₹", maximum_profit)

```

OUTPUT :-

```

Maximum Profit = ₹ 2200.0

```

10. Check whether the available machine and labour hours are satisfied.

CODE :-

```

machine_used = 2 * product_A + 1 * product_B
labour_used = 1 * product_A + 2 * product_B
print("Machine Hours Used:", machine_used)
print("Machine Hours Available:", 100)
print("Labour Hours Used:", labour_used)
print("Labour Hours Available:", 80)

```

OUTPUT :-

```
Machine Hours Used: 100.0
Machine Hours Available: 100
Labour Hours Used: 80.0
Labour Hours Available: 80
```

11. Display the optimal solution in a suitable format.

CODE :-

```
print("----- OPTIMAL SOLUTION -----")
print("Product A:", product_A, "units")
print("Product B:", product_B, "units")
print("Machine Hours Used:", machine_used)
print("Labour Hours Used:", labour_used)
print("Maximum Profit: ₹", maximum_profit)
```

OUTPUT :-

```
----- OPTIMAL SOLUTION -----
Product A: 40.0 units
Product B: 20.0 units
Machine Hours Used: 100.0
Labour Hours Used: 80.0
Maximum Profit: ₹ 2200.0
```

12. Interpret the optimal solution in a business context.

The company should produce 40 units of Product A and 20 units of Product B. This production plan uses all 100 available machine hours and 80 available labour hours. The resulting maximum profit is ₹2,200.

13. Write a conclusion based on the optimization result.

Linear programming helps the business determine the optimal combination of Product A and Product B while considering limited machine and labour resources. The optimal solution is to produce 40 units of Product A and 20 units of Product B, giving a maximum profit of ₹2,200 while satisfying all resource constraints.

Part B: Simulation Modeling

Business Problem: A supermarket wants to decide the appropriate inventory level for a product when daily demand is uncertain.

Practical Questions:

1. Generate random daily demand values for a product for 30 days.
2. Display the generated demand values.

CODE :-

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(42)
```

```
demand = np.random.randint(20, 61, 30)
print("Daily Demand:")
print(demand)
```

OUTPUT :-

```
Daily Demand:
[58 48 34 27 40 58 38 42 30 30 43 55 59 43 22 41 21 43 49 57 21 40 52 31
 41 44 46 47 35 34]
```

3. Select an inventory level of 30 units.
4. Compare the daily demand with the available inventory.
5. Calculate the number of units sold each day.
6. Calculate the number of units left after each day.
7. Calculate the shortage when demand is greater than available inventory.

CODE :-

```
inventory = 30
sold = np.minimum(demand, inventory)
shortage = np.maximum(demand - inventory, 0)
remaining = inventory - sold
print("Units Sold:", sold)
print("Inventory Left:", remaining)
print("Shortage:", shortage)
```

OUTPUT :-

```
Units Sold: [30 30 30 27 30 30 30 30 30 30 30 30 30 30 22 30 21 30 30 30 21 30 30 30
 30 30 30 30 30 30]
Inventory Left: [0 0 0 3 0 0 0 0 0 0 0 0 0 0 8 0 9 0 0 0 9 0 0 0 0 0 0 0]
Shortage: [28 18 4 0 10 28 8 12 0 0 13 25 29 13 0 11 0 13 19 27 0 10 22 1
 11 14 16 17 5 4]
```

8. Calculate the total units sold and total shortage for 30 days.

CODE :-

```
print("Total Units Sold:", sold.sum())
print("Total Shortage:", shortage.sum())
```

OUTPUT :-

```
Total Units Sold: 871
Total Shortage: 358
```

9. Repeat the simulation using inventory levels of 30, 40, and 50 units.
10. Store the results of all three scenarios in a DataFrame.

CODE :-

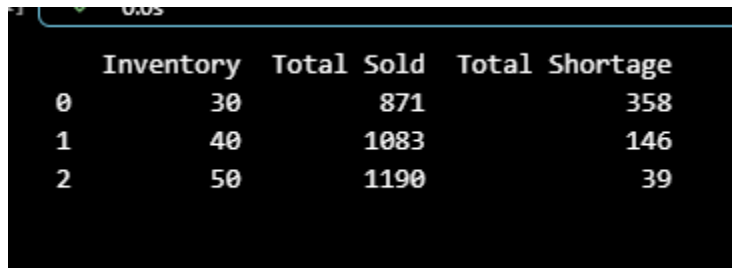
```
results = []
for inventory in [30, 40, 50]:
```

```

sold = np.minimum(demand, inventory)
shortage = np.maximum(demand - inventory, 0)
results.append([
    inventory,
    sold.sum(),
    shortage.sum()
])
results_df = pd.DataFrame(
    results,
    columns=["Inventory", "Total Sold", "Total Shortage"]
)
print(results_df)

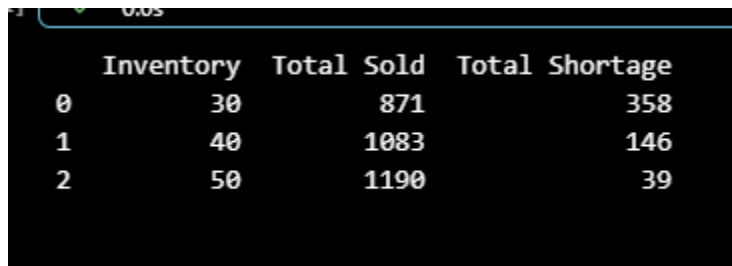
```

OUTPUT :-



	Inventory	Total Sold	Total Shortage
0	30	871	358
1	40	1083	146
2	50	1190	39

11. Compare the total sales and total shortage for each inventory level.



	Inventory	Total Sold	Total Shortage
0	30	871	358
1	40	1083	146
2	50	1190	39

12. Create a chart to compare the results of the three scenarios.

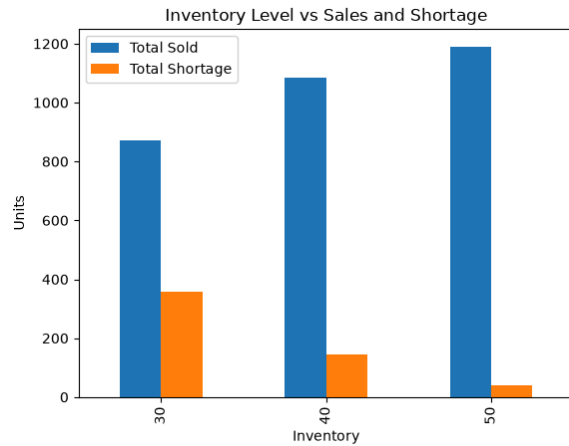
CODE :-

```

results_df.plot(
    x="Inventory",
    y=["Total Sold", "Total Shortage"],
    kind="bar"
)
plt.title("Inventory Level vs Sales and Shortage")
plt.ylabel("Units")
plt.show()

```

OUTPUT :-



13. Identify how changing the inventory level affects sales and shortage.

As inventory level increases, total units sold increase and total shortage decreases.

14. Write a conclusion about the inventory level based on the simulation results.

Increasing inventory helps satisfy more demand and reduces shortages. However, a suitable inventory level should balance product availability with inventory holding costs.